NON-WSQ COURSE

LEARNER GUIDE

For

Arduino Masterclass

Course Code: C218

Conducted by Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

Document Version Control Record

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	30 September 2026	First version	Tertiary Infotech Academy Pte Ltd

Contents

Course Overview

Learning Outcomes

Safety and Toolchain

Topic 1: Introduction to Embedded Systems

- Lab 1: Board Check and Deterministic Blink
- Lab 2: Serial Command Control

Topic 2: Introduction to Basic Electronics

- Lab 3: Safe LED Circuit and Ohm's Law
- Lab 4: Push Button with INPUT_PULLUP
- Lab 5: PWM Fade with Non-blocking Timing

Topic 3: Analogue Sensors and Transducers

- Lab 6: LDR Dark-Activated Lamp with Hysteresis
- Lab 7: Potentiometer-Controlled Servo

Topic 4: Digital Sensors and Transducers

- Lab 8: Ultrasonic Distance with Timeout
- Lab 9: PIR Motion-Activated Light

Topic 5: Actuators

- Lab 10: DHT11 Temperature and Humidity Monitor
- Lab 11: Stepper Motor Position Control
- Lab 12: DHT Fan Controller with Hysteresis

Technical References

Course Overview

This guide develops one continuous embedded-control stack: sense a physical condition, condition and acquire the signal, make a bounded decision, drive a protected actuator, and verify the result with observable evidence. It retains the approved five-topic coverage from the original v12 course while replacing legacy overview-only treatment with current Arduino mechanisms, calculations, code contracts, failure analysis, and acceptance tests.

Course code	C218
Duration	One day: 7.5 instructional hours (9:30am - 5:30pm)
Course page	https://www.tertiarycourses.com.sg/arduino-masterclass.html
Arduino reference	https://docs.arduino.cc/language-reference/

Learning Outcomes

LO1 - Analyse real-life applications using Arduino microcontroller

LO2 - Implement basic electrical circuits using Arduino microcontroller

LO3 - Apply Arduino microcontroller to analogue sensors and transducers

LO4 - Apply Arduino microcontroller to digital sensors and transducers

LO5 - Implement actuators with Arduino microcontroller

Safety and Toolchain

- Disconnect power before rewiring or continuity testing.
- Never power a motor or servo from an Arduino I/O pin. Use a suitable driver and supply.
- Use a flyback diode for inductive loads and establish a common ground.
- Verify board and serial port before upload; use a known USB data cable.
- Keep conductors clear and stop immediately if a component overheats, smells, or the board repeatedly resets.

Topic 1: Introduction to Embedded Systems

An embedded system is a bounded control system. Its architecture, memory, timing, I/O and evidence must support a dedicated physical function. On reset, the bootloader and sketch initialise hardware before loop() executes repeatedly. Serial communication provides both a command path and a diagnostic trace.

Slide alignment

Slides 10-27 | Skill focus: Execute performance specification and analysis of sensors and actuators for real-life applications | Key concepts: Underlying concepts pertaining to performance specification and analysis

Mechanism checklist

- Controller resources and boundaries

- setup()/loop() runtime
- blocking vs non-blocking timing
- UART command framing and evidence

Lab 1: Board Check and Deterministic Blink

Alignment	LO1
Time	15 minutes
Objective	Validate the Arduino Uno/Maker UNO toolchain and prove deterministic digital output timing.
Deliverable	A board-identification record and a 250 ms LED heartbeat that verifies compile, upload, reset, and loop execution.
Folder	labs/lab-01-board-check-blink/

Components

- Arduino Uno or Maker UNO
- USB data cable
- Arduino IDE 2.x

Wiring contract

From	To	Purpose
LED_BUILTIN	On-board LED	No external wiring
USB	Computer	Power + serial upload

Detailed procedure

1. Connect the board using a USB data cable and identify the board/port shown by the IDE.
2. Open the starter sketch and predict the LED period before compiling.
3. Select Arduino Uno and the detected serial port, then compile and upload.
4. Time ten complete blink cycles and compare the observed period with the expected 500 ms cycle.
5. Change INTERVAL_MS to 1000, upload again, and record the changed evidence.

Solution sketch

```
const uint8_t LED_PIN = LED_BUILTIN;
const unsigned long INTERVAL_MS = 250;

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(INTERVAL_MS);
  digitalWrite(LED_PIN, LOW);
  delay(INTERVAL_MS);
}
```

Acceptance checks

- Sketch compiles with no error
- Upload completes on the selected port
- LED cycles 250 ms ON and 250 ms OFF
- Measured ten-cycle duration is approximately 5 seconds

Troubleshooting

Symptom	Diagnostic / correction
Port missing	Try a known USB data cable, reconnect, and confirm the CH340/USB serial device.
Upload timeout	Check Board and Port, close Serial Monitor, then press reset before retrying.
Wrong timing	Confirm milliseconds and that both HIGH and LOW paths use INTERVAL_MS.

Lab 2: Serial Command Control

Alignment	LO1
Time	15 minutes
Objective	Trace UART data from a typed ASCII command to a validated output state.
Deliverable	A line-oriented command parser that accepts ON, OFF, and STATUS without blocking indefinitely.
Folder	labs/lab-02-serial-command-control/

Components

- Arduino Uno or Maker UNO
- USB data cable
- Arduino IDE Serial Monitor

Wiring contract

From	To	Purpose
LED_BUILTIN	On-board LED	Output
USB serial	Computer	9600 bit/s, newline terminator

Detailed procedure

6. Upload the starter sketch and open Serial Monitor at 9600 baud with Newline selected.
7. Send ON, OFF, and STATUS while observing both the LED and acknowledgement text.
8. Send a lowercase command and an invalid command; record how the parser handles each.
9. Disconnect and reconnect the board, then confirm the default output is safe and OFF.

Solution sketch

```
String command;
bool ledOn = false;

void applyOutput() {
  digitalWrite(LED_BUILTIN, ledOn ? HIGH : LOW);
}

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  applyOutput();
  Serial.begin(9600);
  Serial.println("READY: ON | OFF | STATUS");
}

void loop() {
  if (Serial.available() == 0) return;
  command = Serial.readStringUntil('\n');
  command.trim();
  command.toUpperCase();
  if (command == "ON") ledOn = true;
  else if (command == "OFF") ledOn = false;
  else if (command == "STATUS") { /* report only */ }
  else { Serial.println("ERR"); return; }
  applyOutput();
  Serial.println(ledOn ? "LED=ON" : "LED=OFF");
}
```

Acceptance checks

- ON lights the LED
- OFF extinguishes the LED
- STATUS reports the true output state
- Unknown input returns ERR without changing output

Troubleshooting

Symptom	Diagnostic / correction
Garbled text	Set Serial Monitor to 9600 baud to match Serial.begin().
Commands ignored	Select Newline and confirm the parser removes carriage return/newline characters.
LED state wrong	Read back the maintained ledOn state rather than assuming the command succeeded.

Topic 2: Introduction to Basic Electronics

Basic electronics turns code into safe physical behaviour. Ohm's law sets current limits, breadboard topology determines real connectivity, pull resistors prevent floating inputs, debounce rejects switch

transitions, and PWM changes duty cycle while a driver protects the microcontroller from higher-current loads.

Slide alignment

Slides 28-46 | Skill focus: Utilise commonly adopted component interconnections and signal-conditioning principles in automation | Key concepts: Implementation of component interconnections and signal conditioning concepts

Mechanism checklist

- Ohm's law and safe LED current
- breadboard/polarity
- INPUT_PULLUP and debounce
- PWM duty cycle and protected loads

Lab 3: Safe LED Circuit and Ohm's Law

Alignment	LO2
Time	10 minutes
Objective	Calculate and build a current-limited LED circuit within an Arduino digital pin's electrical boundary.
Deliverable	A documented 5 V digital-output circuit using a 330 ohm resistor and measured LED voltage.
Folder	labs/lab-03-safe-led-circuit/

Components

- Arduino Uno or Maker UNO
- Breadboard
- LED
- 330 ohm resistor
- Jumper wires
- Multimeter

Wiring contract

From	To	Purpose
D7	330 ohm resistor	Series current limit
Resistor	LED anode	Long lead
LED cathode	GND	Short lead / flat edge

Detailed procedure

10. Calculate R=(5 V-2 V)/0.01 A and explain why 330 ohm is a safe standard value.
11. Build the series path D7 -> resistor -> LED -> GND with power disconnected.
12. Upload the sketch and verify the LED toggles once per second.
13. Measure LED forward voltage while ON and calculate the observed current from the resistor voltage.

Solution sketch

```
const uint8_t LED_PIN = 7;

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(500);
  digitalWrite(LED_PIN, LOW);
  delay(500);
}
```

Acceptance checks

- LED polarity is correct
- A current-limiting resistor is in series
- LED toggles at 1 Hz
- Calculated current remains close to or below 10 mA

Troubleshooting

Symptom	Diagnostic / correction
LED never lights	Reverse the LED and confirm D7 is connected through the resistor to the anode.
LED very dim	Measure the supply and forward voltage; confirm the resistor is 330 ohm, not 330 kilohm.
Board resets	Disconnect immediately and check for a short between 5 V and GND.

Lab 4: Push Button with INPUT_PULLUP

Alignment	LO2
Time	10 minutes
Objective	Eliminate a floating input and debounce a mechanical switch using a stable time window.
Deliverable	An active-LOW button that toggles an LED once per physical press without double triggers.
Folder	labs/lab-04-button-input-pullup/

Components

- Arduino Uno or Maker UNO
- Push button
- LED circuit from Lab 3
- Jumper wires

Wiring contract

From	To	Purpose
D2	Push button terminal A	INPUT_PULLUP
Push button terminal B	GND	Pressed = LOW
D7	LED + 330 ohm	Output evidence

Detailed procedure

14. Place the button across the breadboard centre gap and identify the internally common legs.
15. Wire D2 to one side and GND to the opposite side; do not add an external pull-down.
16. Upload the sketch and press/release the button repeatedly at different speeds.
17. Observe the raw state in Serial Monitor and confirm one LED toggle per accepted press.

Solution sketch

```
const uint8_t BUTTON_PIN = 2;
const uint8_t LED_PIN = 7;
const unsigned long DEBOUNCE_MS = 30;
bool ledOn = false;
bool stableState = HIGH;
bool lastRaw = HIGH;
unsigned long changedAt = 0;

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  bool raw = digitalRead(BUTTON_PIN);
  if (raw != lastRaw) { changedAt = millis(); lastRaw = raw; }
  if (millis() - changedAt >= DEBOUNCE_MS && raw != stableState) {
    stableState = raw;
    if (stableState == LOW) {
      ledOn = !ledOn;
      digitalWrite(LED_PIN, ledOn ? HIGH : LOW);
    }
  }
}
```

Acceptance checks

- Released input reads HIGH
- Pressed input reads LOW
- LED toggles once per press
- Bounce within 30 ms does not create extra toggles

Troubleshooting

Symptom	Diagnostic / correction
Input always LOW	Check that D2 and GND are not on the same internally connected button leg.
Random triggers	Confirm INPUT_PULLUP is used and all grounds are common.
Double toggles	Increase DEBOUNCE_MS slightly and ensure only the falling edge toggles state.

Lab 5: PWM Fade with Non-blocking Timing

Alignment	LO2
Time	15 minutes
Objective	Relate 8-bit PWM values to duty cycle and vary LED brightness without blocking other logic.
Deliverable	A triangular 0-255 PWM ramp on D9 with calculated duty-cycle checkpoints.
Folder	labs/lab-05-pwm-fade/

Components

- Arduino Uno or Maker UNO
- LED
- 330 ohm resistor
- Breadboard
- Jumper wires

Wiring contract

From	To	Purpose
D9 (~)	330 ohm resistor	PWM output
Resistor	LED anode	Series
LED cathode	GND	Return

Detailed procedure

18. Move the LED circuit to PWM-capable pin D9 and confirm the tilde marking/pin map.
19. Upload the non-blocking fade sketch and monitor the brightness value at key checkpoints.
20. Calculate duty cycle for values 64, 128, and 192 using value/255 x 100%.
21. Change STEP to 15 and compare the smoothness and cycle duration.

Solution sketch

```
const uint8_t LED_PIN = 9;
const unsigned long UPDATE_MS = 20;
const int STEP = 5;
int brightness = 0;
int direction = STEP;
unsigned long lastUpdate = 0;

void setup() { pinMode(LED_PIN, OUTPUT); }

void loop() {
  if (millis() - lastUpdate < UPDATE_MS) return;
  lastUpdate = millis();
  brightness += direction;
  if (brightness >= 255) { brightness = 255; direction = -STEP; }
  if (brightness <= 0) { brightness = 0; direction = STEP; }
  analogWrite(LED_PIN, brightness);
}
```

Acceptance checks

- LED fades up and down continuously
- No delay() is used in loop()
- Value never exceeds 0-255
- Calculated 128 duty cycle is approximately 50%

Troubleshooting

Symptom	Diagnostic / correction
No fade	Confirm the LED is on a PWM pin such as D9, not a non-PWM pin.
Brightness jumps	Reduce STEP or shorten UPDATE_MS.
Value wraps	Reverse direction before the integer crosses the 0 or 255 bounds.

Topic 3: Analogue Sensors and Transducers

Analogue control starts with a continuous quantity and ends with a bounded digital decision. The 10-bit ADC quantises voltage into 0-1023, calibration maps real endpoints, filtering reduces noise, hysteresis stabilises threshold control, and servo power must remain separate from the signal pin.

Slide alignment

Slides 47-64 | Skill focus: Apply analogue sensors and transducers to solve real-world control problems | Key concepts: Applications of analogue sensors and transducers

Mechanism checklist

- ADC conversion and resolution

- calibration/filtering
- LDR divider and hysteresis
- servo mapping and power

Lab 6: LDR Dark-Activated Lamp with Hysteresis

Alignment	LO3
Time	15 minutes
Objective	Convert a light-dependent voltage divider into a stable two-threshold control decision.
Deliverable	A calibrated dark-activated lamp that avoids chatter around the switching threshold.
Folder	labs/lab-06-ldr-dark-lamp/

Components

- Arduino Uno or Maker UNO
- LDR
- 10 kilohm resistor
- LED + 330 ohm
- Breadboard
- Jumper wires

Wiring contract

From	To	Purpose
5 V	LDR	Divider top
LDR/resistor junction	A0	ADC input
10 kilohm resistor	GND	Divider bottom
D7	LED circuit	Lamp output

Detailed procedure

22. Build the divider and record A0 values under bright, room, and covered conditions.
23. Choose ON_THRESHOLD below the dark value and OFF_THRESHOLD above it to create a dead band.
24. Upload the solution and slowly shade the LDR while observing state and ADC evidence.
25. Explain why one threshold would chatter when readings contain noise.

Solution sketch

```
const uint8_t LDR_PIN = A0;
const uint8_t LAMP_PIN = 7;
const int ON_THRESHOLD = 350;
const int OFF_THRESHOLD = 450;
bool lampOn = false;

void setup() {
  pinMode(LAMP_PIN, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int light = analogRead(LDR_PIN);
  if (!lampOn && light < ON_THRESHOLD) lampOn = true;
  if (lampOn && light > OFF_THRESHOLD) lampOn = false;
  digitalWrite(LAMP_PIN, lampOn ? HIGH : LOW);
  Serial.println(light);
  delay(100);
}
```

Acceptance checks

- Serial output shows plausible 0-1023 readings
- Lamp turns ON in dark conditions
- Lamp turns OFF in bright conditions
- Output remains stable inside the hysteresis band

Troubleshooting

Symptom	Diagnostic / correction
Reading fixed at 0	Check A0 is not shorted to GND and the LDR is connected to 5 V.
Reading fixed at 1023	Check the 10 kilohm resistor reaches GND.
Logic reversed	Swap divider orientation or reverse the comparison to match measured calibration.

Lab 7: Potentiometer-Controlled Servo

Alignment	LO3
Time	15 minutes
Objective	Map a 10-bit ADC measurement to a bounded servo command while respecting the servo power boundary.
Deliverable	A 0-180 degree servo position controller with clamped input and separate-power guidance.
Folder	labs/lab-07-pot-servo/

Components

- Arduino Uno or Maker UNO
- 10 kilohm potentiometer
- SG90 servo
- External regulated 5 V supply recommended
- Jumper wires

Wiring contract

From	To	Purpose
Pot outer pins	5 V and GND	Divider
Pot wiper	A0	ADC input
Servo signal	D9	Control pulse
Servo V+	Regulated 5 V	Do not overload I/O pin
All grounds	Common GND	Signal reference

Detailed procedure

26. Wire the potentiometer divider and confirm A0 spans most of 0-1023.
27. Connect the servo signal to D9 and establish a common ground with its power source.
28. Upload the solution, rotate slowly, and verify commanded angle follows the knob.
29. Test the endpoints and confirm constrain() prevents commands outside 0-180 degrees.

Solution sketch

```
#include <Servo.h>
Servo actuator;
const uint8_t POT_PIN = A0;
const uint8_t SERVO_PIN = 9;

void setup() {
  actuator.attach(SERVO_PIN);
  Serial.begin(9600);
}

void loop() {
  int raw = analogRead(POT_PIN);
  int angle = constrain(map(raw, 0, 1023, 0, 180), 0, 180);
  actuator.write(angle);
  Serial.println(angle);
  delay(15);
}
```

Acceptance checks

- ADC reading changes smoothly
- Servo follows the potentiometer
- Command stays within 0-180 degrees
- Board does not reset when servo moves

Troubleshooting

Symptom	Diagnostic / correction
Servo jitters	Improve the 5 V supply, common the grounds, and add a capacitor near the servo.
Servo moves backward	Reverse the map output range if required by the application.
Board resets	Power the servo from a suitable external 5 V source rather than an I/O pin.

Topic 4: Digital Sensors and Transducers

Digital sensors encode information as levels, pulse widths or framed data. Reliable control requires correct thresholds, edges, timeouts and explicit fault states. Ultrasonic distance uses round-trip time of flight; PIR motion produces an active-level contract combined with a non-blocking hold timer.

Slide alignment

Slides 65-83 | Skill focus: Apply digital transducers to solve real-world control problems | Key concepts: Applications of digital transducers

Mechanism checklist

- digital output contracts
- ultrasonic trigger/echo and timeout
- distance formula
- PIR state and hold timing

Lab 8: Ultrasonic Distance with Timeout

Alignment	LO4
Time	15 minutes
Objective	Measure echo pulse width, convert time-of-flight to distance, and handle a missing echo safely.
Deliverable	A distance monitor that reports centimetres or NO_ECHO within a bounded 30 ms wait.
Folder	labs/lab-08-ultrasonic-distance/

Components

- Arduino Uno or Maker UNO
- HC-SR04/HY-SRF05 ultrasonic sensor
- Breadboard
- Jumper wires

Wiring contract

From	To	Purpose
Sensor VCC	5 V	Supply
Sensor GND	GND	Return
TRIG	D9	10 microsecond output pulse
ECHO	D10	Pulse input

Detailed procedure

30. Wire the sensor and confirm the trigger and echo pins are not swapped.
31. Upload the sketch and position a flat target 20-100 cm in front of the sensor.
32. Compare Serial Monitor distance with a ruler at three positions.
33. Remove the target and confirm the 30 ms timeout returns NO_ECHO instead of blocking forever.

Solution sketch

```
const uint8_t TRIG_PIN = 9;
const uint8_t ECHO_PIN = 10;
const unsigned long ECHO_TIMEOUT_US = 30000;

void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  unsigned long duration = pulseIn(ECHO_PIN, HIGH, ECHO_TIMEOUT_US);
  if (duration == 0) Serial.println("NO_ECHO");
  else {
    float distanceCm = duration * 0.0343f / 2.0f;
    Serial.println(distanceCm, 1);
  }
  delay(100);
}
```

Acceptance checks

- Distance is within a reasonable tolerance of the ruler
- Readings update continuously
- Missing echo returns NO_ECHO
- No divide-by-zero or indefinite wait occurs

Troubleshooting

Symptom	Diagnostic / correction
Always NO_ECHO	Swap TRIG/ECHO if reversed and verify the sensor has 5 V and common GND.
Distance doubles	The conversion must divide the round-trip distance by two.
Unstable values	Use a flat target, avoid soft fabric, and keep within the sensor's working zone.

Lab 9: PIR Motion-Activated Light

Alignment	LO4
Time	10 minutes
Objective	Use a digital motion sensor contract and a non-blocking hold timer to control a light.
Deliverable	A motion light that extends its ON period when new motion arrives and expires safely after five seconds.
Folder	labs/lab-09-pir-motion-light/

Components

- Arduino Uno or Maker UNO
- HC-SR501 PIR sensor
- LED + 330 ohm
- Breadboard
- Jumper wires

Wiring contract

From	To	Purpose
PIR VCC	5 V	Supply
PIR GND	GND	Return
PIR OUT	D2	Digital input
D7	LED circuit	Light output

Detailed procedure

34. Wire the PIR and allow its warm-up/calibration interval after power-up.
35. Upload the sketch and move across the sensor field of view.
36. Trigger motion again before five seconds expires and confirm the hold timer extends.
37. Remain still and verify the output turns OFF after the bounded hold period.

Solution sketch

```
const uint8_t PIR_PIN = 2;
const uint8_t LIGHT_PIN = 7;
const unsigned long HOLD_MS = 5000;
unsigned long lastMotionMs = 0;

void setup() {
  pinMode(PIR_PIN, INPUT);
  pinMode(LIGHT_PIN, OUTPUT);
}

void loop() {
  if (digitalRead(PIR_PIN) == HIGH) lastMotionMs = millis();
  bool active = lastMotionMs != 0 && millis() - lastMotionMs < HOLD_MS;
  digitalWrite(LIGHT_PIN, active ? HIGH : LOW);
}
```

Acceptance checks

- Motion drives the output HIGH
- Output remains ON for five seconds after the latest motion
- New motion extends the timer
- loop() remains responsive without delay(5000)

Troubleshooting

Symptom	Diagnostic / correction
Always HIGH	Wait for warm-up and check the sensor retrigger jumper and sensitivity settings.
No detection	Confirm OUT reaches D2 and walk laterally across the field of view.
Light turns off too soon	Update lastMotionMs every time the PIR is HIGH.

Topic 5: Actuators

Actuators consume energy and therefore require a power stage. MOSFETs or driver arrays switch load current, flyback paths clamp inductive energy, step sequences create position, and feedback controllers validate sensor readings before applying hysteresis and commanding the actuator.

Slide alignment

Slides 84-102 | Skill focus: Develop actuator networks with stepper and continuous-drive actuators | Key concepts: Actuator networks

Mechanism checklist

- driver energy boundary
- DC motor/flyback protection
- stepper phases and position

- DHT validity and feedback hysteresis

Lab 10: DHT11 Temperature and Humidity Monitor

Alignment	LO5
Time	10 minutes
Objective	Read a library-managed digital sensor transaction and reject invalid measurements before control use.
Deliverable	A one-hertz temperature/humidity monitor that identifies NaN failures instead of treating them as real values.
Folder	labs/lab-10-dht-monitor/

Components

- Arduino Uno or Maker UNO
- DHT11 module
- 10 kilohm pull-up if using a bare sensor
- Jumper wires
- Adafruit DHT sensor library

Wiring contract

From	To	Purpose
DHT VCC	5 V	Supply
DHT GND	GND	Return
DHT DATA	D2	Digital data
DATA to VCC	10 kilohm	Pull-up for bare sensor

Detailed procedure

38. Install the Adafruit DHT sensor library and its required dependency from Library Manager.
39. Wire the DHT11 and upload the monitor sketch.
40. Warm the sensor gently and observe the change without touching exposed conductors.
41. Disconnect DATA briefly and confirm the sketch reports READ_ERROR rather than a numeric control value.

Solution sketch

```
#include <DHT.h>
const uint8_t DHT_PIN = 2;
#define DHT_TYPE DHT11
DHT dht(DHT_PIN, DHT_TYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  if (isnan(t) || isnan(h)) Serial.println("READ_ERROR");
  else {
    Serial.print("T_C="); Serial.print(t, 1);
    Serial.print(",RH="); Serial.println(h, 1);
  }
  delay(2000);
}
```

Acceptance checks

- Temperature and humidity update approximately once per second
- Values are plausible for the room
- Disconnected/failed read reports READ_ERROR
- dht.begin() is called in setup()

Troubleshooting

Symptom	Diagnostic / correction
READ_ERROR	Check sensor type, D2 wiring, pull-up, common ground, and the two-second minimum sample interval if needed.
Values never change	Avoid reading faster than the sensor supports and confirm the correct DHT type.
Compile error	Install both the DHT sensor library and Adafruit Unified Sensor dependency.

Lab 11: Stepper Motor Position Control

Alignment	LO5
Time	10 minutes
Objective	Drive a four-phase stepper through a transistor array and relate commanded steps to angular position.
Deliverable	A 28BYJ-48/ULN2003 actuator that moves one revolution forward and backward at a bounded speed.
Folder	labs/lab-11-stepper-position/

Components

- Arduino Uno or Maker UNO
- 28BYJ-48 stepper
- ULN2003 driver board
- External 5 V supply
- Jumper wires

Wiring contract

From	To	Purpose
D8-D11	ULN2003 IN1-IN4	Phase controls
ULN2003 5 V	External 5 V	Motor power
ULN2003 GND	Arduino GND	Common reference
Stepper	ULN2003 socket	Keyed connector

Detailed procedure

42. Connect the driver inputs in the library sequence and establish common ground.
43. Upload the sketch with STEPS_PER_REV set for the supplied motor/gearbox.
44. Mark the shaft, observe one forward and one reverse revolution, and record position error.
45. Increase speed modestly and identify the point where the motor begins to miss steps.

Solution sketch

```
#include <Stepper.h>
const int STEPS_PER_REV = 2048;
Stepper motor(STEPS_PER_REV, 8, 10, 9, 11);

void setup() {
  motor.setSpeed(10);
}

void loop() {
  motor.step(STEPS_PER_REV);
  delay(500);
  motor.step(-STEPS_PER_REV);
  delay(500);
}
```

Acceptance checks

- Motor completes a forward and reverse move
- Direction reverses correctly
- Driver—not Arduino I/O—supplies coil current
- Grounds are common and board remains stable

Troubleshooting

Symptom	Diagnostic / correction
Motor vibrates	Check IN1-IN4 order and use the phase order required by the library/driver.
Weak torque	Use a suitable external 5 V supply and lower the speed.
Wrong angle	Calibrate STEPS_PER_REV for the actual gearbox and stepping mode.

Lab 12: DHT Fan Controller with Hysteresis

Alignment	LO5
Time	10 minutes
Objective	Integrate measurement, decision logic, and a protected continuous-drive actuator into a feedback controller.
Deliverable	A temperature-controlled DC fan with MOSFET switching, flyback protection, PWM drive, and hysteresis.
Folder	labs/lab-12-dht-fan-controller/

Components

- Arduino Uno or Maker UNO
- DHT11
- Logic-level N-channel MOSFET
- DC fan/motor
- Flyback diode
- External motor supply
- Jumper wires

Wiring contract

From	To	Purpose
DHT DATA	D2	Measurement
D5 PWM	MOSFET gate through 220 ohm	Control
10 kilohm	Gate to GND	Safe default OFF
Motor/fan	External supply + MOSFET drain	Power path
Flyback diode	Across motor	Cathode to supply +
All grounds	Common GND	Reference

Detailed procedure

46. Build the low-side MOSFET driver with power disconnected and verify diode polarity.
47. Upload the controller and confirm the fan is OFF below FAN_ON_C.
48. Warm the DHT11 above the ON threshold and observe full PWM drive.
49. Let the sensor cool below FAN_OFF_C and confirm the fan does not chatter between thresholds.

Solution sketch

```
#include <DHT.h>
const uint8_t DHT_PIN = 2;
const uint8_t FAN_PWM_PIN = 5;
const float FAN_ON_C = 30.0;
const float FAN_OFF_C = 28.0;
DHT dht(DHT_PIN, DHT11);
bool fanOn = false;

void setup() {
  pinMode(FAN_PWM_PIN, OUTPUT);
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float t = dht.readTemperature();
  if (isnan(t)) { Serial.println("READ_ERROR"); delay(2000); return; }
  if (!fanOn && t >= FAN_ON_C) fanOn = true;
  if (fanOn && t <= FAN_OFF_C) fanOn = false;
  analogWrite(FAN_PWM_PIN, fanOn ? 255 : 0);
  Serial.print("T_C="); Serial.print(t, 1);
  Serial.print(",FAN="); Serial.println(fanOn ? "ON" : "OFF");
  delay(2000);
}
```

Acceptance checks

- Fan turns ON above 30 C
- Fan remains ON until temperature falls below 28 C
- Invalid DHT read leaves the previous safe state and reports an error
- Motor current never flows through an Arduino I/O pin

Troubleshooting

Symptom	Diagnostic / correction
Fan never starts	Check external supply, common ground, MOSFET pinout, and gate drive.
Arduino resets	Separate motor power, add decoupling, and verify flyback diode orientation.
Fan chatters	Use distinct ON/OFF thresholds and a realistic sample interval.

Technical References

Arduino language reference: https://docs.arduino.cc/language-reference/

Arduino PWM Help Center: https://support.arduino.cc/hc/en-us/articles/9350537961500-Use-PWM-output-with-Arduino

Course registration:https://www.tertiarycourses.com.sg/arduino-masterclass.html

Legacy source deck: Reference v12 PPTX supplied with this project